[2026-1] AI프로그래밍

5주차: 데이터 처리 라이브러리

2026.04.03 (금)

김현준

데이터 처리를 왜 먼저 배우는가

모델보다 먼저, 데이터를 다루는 능력이 중요

- 모델 성능은 데이터 품질에 달려 있음
- 오늘은 NumPy 중심, Pandas 기초까지 다룸

데이터 수집 → 정리 → 변환 → 분석 → 모델 입력

NumPy가 필요한 이유

- NumPy는 수치 연산 패키지
- 리스트보다 계산이 간단하고 빠름
- 데이터 전처리와 행렬 계산에 자주 사용

```
# pip install numpy
import numpy as np
```

Python list

- 반복문 필요
- 느림
- 코드 복잡

NumPy array

- 벡터 연산
- 빠름
- 코드 간결

Python 리스트 vs NumPy 배열

같은 연산도 NumPy를 사용하면 더 간단하게 표현됨 — 반복문 없이 한 번에 계산 가능

Python list

```
data = [1, 2, 3, 4]
result = []
for i in data:
    result.append(i * 10)
print(result)
```

NumPy array (Scalar Broadcasting)

```
import numpy as np
arr = np.array([1, 2, 3, 4])
new_arr = arr * 10
print(new_arr)
```

ndarray의 기본 속성

- shape — 배열의 모양 (행, 열, ...)
- ndim — 차원 수
- dtype — 데이터 타입 https://numpy.org/doc/stable/user/basics.types.html

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[1, 2, 3], [4, 5, 6]])

print(a.shape)   # (3,)
print(a.ndim)    # 1
print(a.dtype)   # int32

print(b.shape)   # (2, 3)
print(b.ndim)    # 2
print(b.dtype)   # int32
```

1차원 배열 a: [1, 2, 3]

→ shape (3,)

ndim 1

2차원 배열 b: [[1,2,3],[4,5,6]]

→ shape (2,3) ndim 2

배열 생성 — zeros, ones, full, empty

- 특정 값으로 채워진 배열을 빠르게 생성
- full()은 원하는 값으로 채움
- empty()는 초기화 없이 메모리만 할당 (빠름)

```
import numpy as np

a = np.zeros(5)
b = np.zeros((2, 3))
c = np.ones((2, 3))
d = np.full((2, 3), 7)
e = np.empty((2, 3))

print(a)
print(b)
print(c)
print(d)
print(e)
```

zeros(5) → [0. 0. 0. 0. 0.]

ones((2,3)) → 2×3, 전부 1.0

full((2,3), 7) → 2×3, 전부 7

empty((2,3)) → 2×3, 임의값 (초기화 없음)

shape를 첫 argument로 줌

두번째 argument는 full의 경우 채울 값

numpy.org

Array creation routines — NumPy v2.3 Manual

Return a new array of given shape and type, without initializing entries.

배열 생성 — zeros_like, ones_like, full_like

- 기존 배열과 같은 shape/dtype으로 새 배열 생성
- 원본 배열을 참고해 초기값만 바꿀 때 유용
- eye()는 단위 행렬(대각선이 1) 생성

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

z_like = np.zeros_like(arr)
o_like = np.ones_like(arr)
f_like = np.full_like(arr, 9)
e_like = np.empty_like(arr)
eye_mat = np.eye(3)

print(z_like)
print(o_like)
print(f_like)
print(e_like)
print(eye_mat)
```

zeros_like(arr) → arr와 같은 shape, 전부 0

ones_like(arr) → arr와 같은 shape, 전부 1

full_like(arr, 9) → arr와 같은 shape, 전부 9

empty_like(arr) → arr와 같은 shape, 초기화 없음

eye(3) → 3×3 단위 행렬

배열 생성 — arange, linspace, astype

- arange — 간격(step) 기준으로 생성
- linspace — 개수 기준으로 균등 분할
- astype() — 배열의 dtype 변환

```
import numpy as np

a = np.arange(0, 10, 2)
b = np.arange(12)
c = np.linspace(0, 1, 5)
d = np.linspace(0, 2*3.14159, 100)
e = np.array([1.2, 3.7, -2.4]).astype(int)

print(a)   # [0 2 4 6 8]
print(b)
print(c)   # [0.   0.25 0.5  0.75 1.  ]
print(d)
print(e)   # [ 1  3 -2]
```

arange(0, 10, 2) → [0, 2, 4, 6, 8]

(start, end, step 기준)

linspace(0, 1, 5) → [0.0, 0.25, 0.5, 0.75, 1.0]

(start, end, end 포함해서 갯수 기준)

astype(int) → 소수점 버림 변환 [1, 3, -2]

배열 연산의 기본

- 원소별(element-wise) 덧셈, 곱셈
- 스칼라와의 연산
- NumPy 연산은 기본적으로 element-wise

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)   # [5 7 9]
print(a * b)   # [4 10 18]
print(a - 1)   # [0 1 2]
print(a / 2)   # [0.5 1.  1.5]
```

1

a: [1, 2, 3]

2

b: [4, 5, 6]

3

a + b: [5, 7, 9]

합계와 평균 — axis 기준으로 보기

- axis=0 → 열 방향(위→아래) 집계
- axis=1 → 행 방향(왼→오른) 집계
- 데이터 처리에서 매우 자주 쓰임

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(np.sum(arr)        # 21
print(np.sum(arr, axis=0))  # [5 7 9]
print(np.sum(arr, axis=1))  # [6 15]
print(np.mean(arr, axis=0)) # [2.5 3.5
4.5]
```

axis=0 (열 방향): 각 열의 합 → [5, 7, 9]

row가 dim 0 → 해당 dim이 없어진다고 보면 됨

axis=1 (행 방향): 각 행의 합 → [6, 15]

column이 dim 1 → 해당 dim이 없어진다고 보면 됨

인덱싱 기초

- arr[row, col] 형태로 접근
- : 사용 - 행 전체 선택, 열 전체 선택
- 1차원과 2차원 인덱싱 차이

```
import numpy as np

arr = np.array([[10, 20, 30],
                [40, 50, 60]])

print(arr[0, 1])   # 20
print(arr[1])      # [40 50 60]
print(arr[:, 1])   # [20 50]
```

arr[0, 1] → 0 행 1열 원소: 20

arr[1] → 1행 전체: [40, 50, 60]

arr[:, 1] → 1열 전체: [20, 50]

슬라이싱 기초

- 시작:끝 형태 (끝 인덱스 미포함)
- 행과 열을 함께 자를 수 있음

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

print(arr[:2, 1:])   # [[2 3] [5 6]]
print(arr[1:, :2])   # [[4 5] [7 8]]
print(arr[2, :])    # [7 8 9]
```

arr[:2, 1:] → 상위 2행, 1열 이후 → [[2,3], [5,6]]

arr[1:, :2] → 1행 이후, 처음 2열 → [[4,5], [7,8]]

arr[2, :] → 마지막 행 전체 → [7,8,9]

조건으로 선택하기 — boolean indexing

- 조건식으로 True/False 마스크 생성
- 마스크로 필요한 값만 추출 가능
- 데이터 필터링의 가장 기본적인 방식

```
import numpy as np

arr = np.array([10, 25, 30, 12, 40])

mask = arr >= 25
print(mask)          # [F T T F T]
print(arr[mask])     # [25 30 40]
print(arr[arr % 2 == 0])  # [10 30 12 40]
```

원본: [10, 25, 30, 12, 40]

마스크(>=25): [F, T, T, F, T]

결과: [25, 30, 40]

실습 1 — 조건 선택과 슬라이싱

다음 배열에서 아래 작업을 수행하시오.

1. 두 번째 열을 추출하여 출력
2. 20보다 큰 값들만 추출하여 출력
3. 첫 두 행, 마지막 두 열만 잘라서 출력

```
import numpy as np

arr = np.array([
    [ 5, 10, 15, 20],
    [25, 30, 35, 40],
    [45, 50, 55, 60]
])
```

실습 1 — 예시 정답

```
import numpy as np

arr = np.array([
    [ 5, 10, 15, 20],
    [25, 30, 35, 40],
    [45, 50, 55, 60]
])

second_col = arr[:, 1]
print(second_col)      # [10 30 50]

filtered = arr[arr > 20]
print(filtered)        # [25 30 35 40 45 50 55 60]

part = arr[:2, -2:]
print(part)            # [[15 20] [35 40]]
```

열 추출, 조건 필터링, 2차원 슬라이싱은 NumPy 데이터 처리의 기본 패턴

view와 copy

- 슬라이싱 결과는 원본을 참조(view)할 수 있음
- 안전하게 분리하려면 copy() 필요

```
import numpy as np

arr = np.array([0, 1, 2, 3, 4, 5])

part_view = arr[2:4]
part_copy = arr[2:4].copy()

part_view[0] = 100
print(arr)      # [  0   1 100   3   4   5]

part_copy[0] = 999
print(arr)      # 변화 없음
```

part_view

원본 arr와 연결 → part_view 수정 시 arr도 변경됨

copy()

원본과 완전 분리 → part_copy 수정해도 arr 불변

reshape와 transpose

- reshape — 배열 모양 바꾸기 (-1은 자동 계산)
- transpose / .T — 행과 열 축을 바꿈

```
import numpy as np

arr = np.arange(12)
a = arr.reshape((3, 4))
b = a.T
c = arr.reshape((-1, 4))

print(a)      # shape (3, 4)
print(b)      # shape (4, 3)
print(c)      # shape (3, 4)
```

arange(12): [0,1,...,11] → shape (12,)

reshape((3,4)): 3행 4열 배열 → shape (3,4)

.T: 전치 → shape (4,3)

broadcasting

- 크기가 다른 배열도 규칙이 맞으면 계산 가능
- 작은 배열이 큰 배열에 맞게 자동 확장됨
- NumPy의 핵심 개념 중 하나

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 20, 30])

print(a + b)
# [[11 22 33]
#  [14 25 36]]
```

a: shape (2,3) — 2행 3열

b: shape (3,) → 각 행에 자동 확장

a + b: [[11,22,33],[14,25,36]]

벡터 내적 (dot product)

- 같은 길이의 두 벡터를 원소별로 곱한 뒤 합산
- np.dot(a, b)
- 결과는 스칼라(숫자 하나)

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = np.dot(a, b)
print(result)   # 32
# 1×4 + 2×5 + 3×6 = 4 + 10 + 18 = 32

# 동일한 결과 나옴
print(a @ b)    # 32
print(np.matmul(a, b)) # 32
# batch를 고려하면 @, matmul
```

a: [1, 2, 3] · b: [4, 5, 6]

원소별 곱: [1×4, 2×5, 3×6] = [4, 10, 18]

합산: 4 + 10 + 18 = 32 (스칼라)

행렬 곱 (dot product)

- 행렬의 경우 dot()은 행렬 곱 계산에 사용
- element-wise 곱(*)과 완전히 다름
- (m,n) · (n,k) → (m,k) — shape이 맞아야 함

```
import numpy as np

x = np.array([[1, 2, 3]])   # shape (1,3)
y = np.array([[6, 23, -1]]) # shape (1,3)

print(x.dot(y.T))   # [[49]]
# 1×6 + 2×23 + 3×(-1) = 49
print(x * y)
# [[ 6 46 -3]]
```

x: shape (1, 3)

y.T: shape (3, 1)

x.dot(y.T): shape (1, 1) → [[49]]

실습 2 — reshape, broadcasting, dot

아래 작업을 순서대로 수행하시오.

1. 0부터 11까지의 값을 3×4 배열로 reshape
2. 각 열에 [100, 200, 300, 400]을 broadcasting으로 더하고 출력
3. x와 y의 내적을 계산하고 출력

```
import numpy as np

arr = np.arange(12)
v = np.array([100, 200, 300, 400])

x = np.array([[1, 2, 3]])
y = np.array([[4, 5, 6]])
```

실습 2 예시 정답

```
import numpy as np

arr = np.arange(12)
a = arr.reshape((3, 4))
print(a)

v = np.array([100, 200, 300, 400])
b = a + v
print(b)

x = np.array([[1, 2, 3]])
y = np.array([[4, 5, 6]])
z = x.dot(y.T)
print(z)   # [[32]]
```

reshape로 구조를 바꾸고, broadcasting으로 열별 값을 더하고, dot으로 내적을 계산함

Pandas가 필요한 이유

- Pandas는 표 형태 데이터 처리에 강함
- Series — 1차원 (index + values)
- DataFrame — 2차원 표 (행 + 열)
- CSV 파일을 읽고 다루기에 편리함

```
import pandas as pd
```

Series

1차원 / index + value 쌍 / 예: 월:100, 화:200, 수:300

DataFrame

2차원 / 행 (index) + 열 (column) / 엑셀 표와 유사

Series 만들기

- 1차원 데이터 구조 (Index + Value)
- 다양한 데이터 타입 (숫자, 문자열, 불리언 등) 저장 가능
- 자동으로 인덱스 생성 가능, 또는 수동 지정 가능

```
import pandas as pd

# 리스트로 Series 생성 (자동 인덱스)
s1 = pd.Series([10, 20, 30, 40])
print(s1)

# 인덱스를 직접 지정하여 Series 생성
s2 = pd.Series([100, 200, 300], index=['A', 'B', 'C'])
print(s2)

# 딕셔너리로 Series 생성
data = {'서울': 990, '부산': 340, '대구': 240}
s3 = pd.Series(data)
print(s3)
```

s1: 자동 인덱스 [0,1,2,3]

기본적으로 0부터 시작하는 숫자 인덱스가 부여됩니다.

s2: 사용자 지정 인덱스 ['A','B','C']

데이터의 의미를 부여하는 인덱스를 직접 설정할 수 있습니다.

s3: 딕셔너리 키가 인덱스가 됨

딕셔너리로 생성 시 키가 인덱스, 값이 Series의 값이 됩니다.

DataFrame 만들기

- 딕셔너리로 DataFrame 생성
- index — 행 이름 / columns — 열 이름

```
import pandas as pd

data = {
    "종가": [157000, 51300, 68800, 140000],
    "PER": [39.88, 8.52, 10.03, 228.38],
    "PBR": [4.38, 1.45, 0.87, 2.16]
}
index = ["NAVER", "삼성전자", "LG전자", "카카오"]

df = pd.DataFrame(data, index=index)
print(df)
```

	종가	PER	PBR
NAVER	157000	39.88	4.38
삼성전자	51300	8.52	1.45
LG전자	68800	10.03	0.87
카카오	140000	228.38	2.16

열 선택, 행 선택, 조건 필터링

- df["종가"] — 열 선택
- df.iloc[0] — 위치 기반 행 선택
- df.loc["NAVER"] — 레이블 기반 행 선택
- 조건식으로 필요한 행만 선택

```
print(df["종가"])
print(df.iloc[0])
print(df.loc["NAVER"])
print(df[df["PER"] > 10])
```

df["종가"]: 종가 열 전체

df.iloc[0]: 첫 번째 행 (NAVER)

df.loc["NAVER"]: NAVER 행 (레이블)

df[df["PER"]> 10]: PER이 10 초과인 행만

CSV 파일 읽기와 기본 확인

- read_csv()로 파일 읽기
- head(), shape, columns로 구조 확인
- 특정 열과 특정 행 접근

```
import pandas as pd

df = pd.read_csv("card.csv")

print(df.head())     # 상위 5행 미리보기
print(df.shape)      # (행 수, 열 수) 튜플
print(df.columns)    # 열 이름 목록
print(df["이용일자"])  # 특정 열 전체 선택
print(df.iloc[0])    # 첫 번째 행 선택 (위치 기반)
```

card.csv 파일 읽기

head() / shape / columns 로 구조 파악

열/행 선택으로 원하는 데이터 접근

전처리 기초

- fillna() — 결측치 채우기
- astype() — 형변환
- 새 열 만들기
- sort_values() — 정렬

```
import pandas as pd

df = pd.DataFrame({
    "이름": ["민수", "지연", "현우"],
    "점수": [90, None, 78],
    "반": ["A", "A", "B"]
})

df["점수"] = df["점수"].fillna(0)
df["점수"] = df["점수"].astype(int)
df["통과"] = df["점수"] >= 80

sorted_df = df.sort_values("점수", ascending=False)
# 내림차순
print(sorted_df)
```

전처리 전

이름	점수	반
민수	90	A
지연	NaN	A
현우	78	B

전처리 후

이름	점수	반	통과
민수	90	A	True
현우	78	B	False
지연	0	A	False

평균 열 만들기

- 두 시험 점수의 평균 열 avg 생성
- avg 기준으로 상위 학생 (평균 80점 이상) 필터링 → 내림차순으로 정렬

```
import pandas as pd

df = pd.DataFrame({
    "이름": ["민수", "지연", "현우", "수빈"],
    "중간": [88, 76, 93, 64],
    "기말": [92, 81, 89, 70]
})

df["avg"] = (df["중간"] + df["기말"]) / 2
result = df[df["avg"] >= 80]
result = result.sort_values("avg", ascending=False)

print(result[["이름", "avg"]])
```

avg 열 생성: (중간 + 기말) / 2

조건 필터: avg >= 80

내림차순 정렬

이름, avg 열만 출력

도전 실습 — 제공된 CSV 처리하기

제공된 sales.csv 파일을 읽어 아래 작업을 수행하시오.

1. category가 "food"인 행만 추출
2. month별 amount 합계를 계산
3. 결과를 month 오름차순으로 정리
4. 최종 결과 DataFrame을 출력

※ sales.csv 파일은 별도로 제공됨